SCIENCEUTSAV®
Science is awesome

I'm a ROBO SCIENTIST

Smart Switching and Door Sensor Automation

Session 4

SMART SWITCHING AND DOOR SENSOR AUTOMATION

Curiosity Kickoff

- What happens if we mix all three (red + green + blue)?
- Can we make purple using RGB?
- Why do RGB lights change colors?

Concept Connection

Today, let's meet three special kinds of lights hiding inside your kit. Think of them as the "eyes" of you

This topic introduces students to Push Button Digital Input through the session RGB Control. The goal is to keep the learning simple, practical, and beginner-friendly. Students first understand the concept in easy words, then see a small Arduino program, and then connect the code to the real output or input behavior.

Watt LED PCB

1 Watt LED is a high-brightness light-emitting diode that produces much stronger light compared to normal small LEDs.

DIP LED

A DIP LED (Dual In-line Package LED) is a type of LED with two long metal pins used for easy connection on breadboards and PCB circuits.

Buzzer

A buzzer is an electronic component that generates sound when electric current flows through it. It is commonly utilized in alarms, alerts, and notification systems to provide auditory signals.

toggle switch

- A toggle switch is a switch that stays in its selected position (ON or OFF) until it is manually changed.
- It is commonly used to control circuits and devices by switching the power flow permanently until toggled again.

limit switch

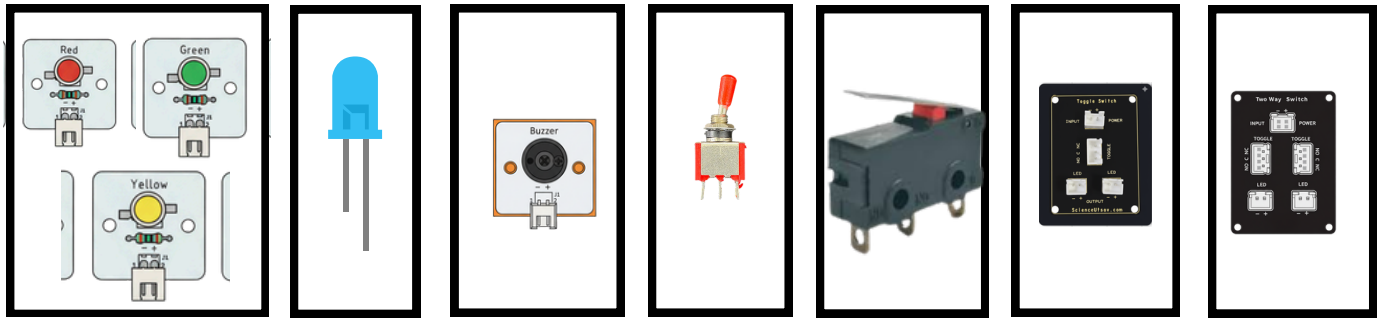
A limit switch is a mechanical switch that gets activated when an object touches or presses it. It is used to detect movement, position, or opening/closing actions in machines and automation projects.

toggle switch PCB

A toggle switch PCB is a small circuit board made to mount and connect a toggle switch neatly in an electronic circuit.

two-way switch PCB

A two-way switch PCB is a circuit board designed to connect two switches that can control the same light or device from two different locations.



Beginner: Limit Switch basic

Class Explanation

In this class, students learn the basic idea of Limit Switch Basics. The code below is kept simple so beginners can focus on understanding the main concept without extra complexity.

Materials Required

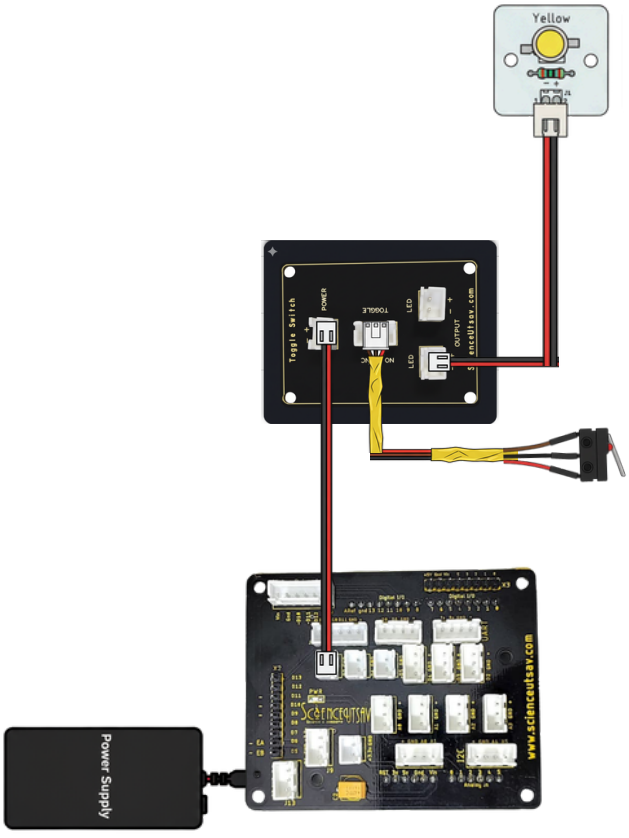
- Arduino with attached shield
- 1 × toggle switch PCB
- 1 × Limit Switch
- 1 × Yellow 1W LED
- RMC wires
- USB cable

ow to Connect

Connect using RMC wires :

- Toggle switch PCB → Arduino pin D7 and GND
- Yellow 1W LED → in Toggle switch PCB connect to led pin and GND
- Limit Switch → in Toggle switch PCB connect to toggle pin and GND

- **“Refer to the link or QR at the back of the book to get the code file for this project.”**



Code

```
/*
Simple Arduino Code
Pin logic based on updated component list and custom switch PCB logic.
*/

const int limitSwitch = 7;

void setup() {
 pinMode(limitSwitch, OUTPUT);
}

void loop() {
 digitalWrite(limitSwitch, HIGH);
}
```

Code Explanation

- In an Arduino program, first we define the pins or variables. This means we give a name to a pin number so it is easy to use in the code. For example, if we write int ledPin = 5, it means we are using pin number 5 and calling it ledPin.
- Next comes the setup() function. This part runs only one time when the Arduino is turned on. It is used to prepare the pins and start things like the Serial Monitor. Inside setup, we use pinMode() to tell the Arduino what each pin will do. If we write pinMode(ledPin, OUTPUT);, it means the pin will send signals to control devices like an LED.
- After that comes the loop() function. This part runs again and again forever. Arduino keeps repeating whatever is written inside loop, so it can continuously control or check the circuit.
- Inside the loop, we can use digitalWrite() to turn an output ON or OFF. For example, digitalWrite(ledPin, HIGH); will turn the LED ON, and digitalWrite(ledPin, LOW); will turn it OFF.

So overall, the program first sets everything in setup(), then keeps running the loop continuously, controlling devices like LEDs using functions such as pinMode() and digitalWrite().

Try This Yourself Code

This Try This Yourself code keeps the same topic of RGB LED Basics but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
const int redPin = 5;
const int greenPin = 6;
const int bluePin = 7;

void setup() \
 pinMode(redPin, OUTPUT);
 pinMode(greenPin, OUTPUT);
 pinMode(bluePin, OUTPUT);
\

void loop() \
 digitalWrite(redPin, HIGH);
 digitalWrite(greenPin, HIGH);
 digitalWrite(bluePin, LOW);
 delay(1000);
 digitalWrite(redPin, LOW);
 digitalWrite(greenPin, HIGH);
 digitalWrite(bluePin, HIGH);
 delay(1000);
 digitalWrite(redPin, HIGH);
 digitalWrite(greenPin, LOW);
 digitalWrite(bluePin, HIGH);
 delay(1000);
\
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learn

- the basic idea of RGB LED Basics how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Intermediate: Mood Lamp

Class Explanation

In this class, students learn the basic idea of RGB LED Basics. The code below is kept simple so beginners can focus on understanding the main concept without extra complexity.

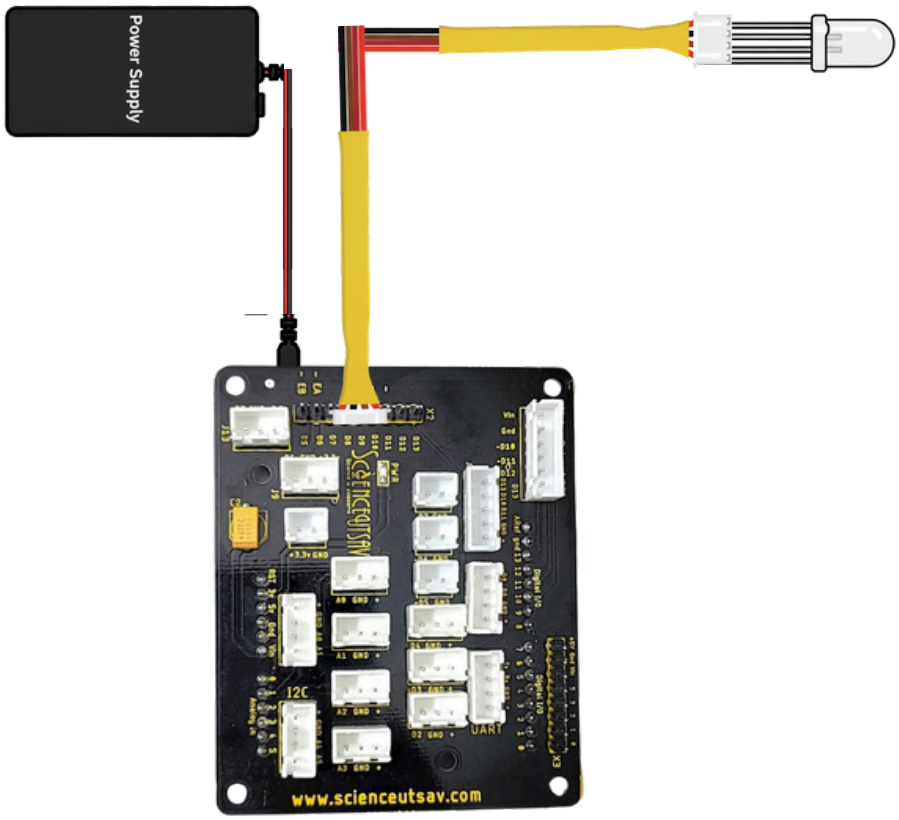
Materials Required

- Arduino with attached shield
- 1 × 4-Leg RGB LED on breakout board
- RMC wires
- USB cable

ow to Connect

- Attach the 4-PIN RMC to Break out pins D5,D6,D7
- Connect using RMC wires with LED :
  - Red leg → Arduino pin D5
  - Green leg → Arduino pin D6
  - Blue leg → Arduino pin D7
- Plug the Arduino into your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
const int redPin = 5;
const int greenPin = 6;
const int bluePin = 7;
void setup() \
 pinMode(redPin, OUTPUT);
 pinMode(greenPin, OUTPUT);
 pinMode(bluePin, OUTPUT);
\
void loop() \
 analogWrite(redPin, 255);
 analogWrite(greenPin, 50);
 digitalWrite(bluePin, LOW);
 delay(1000);

 analogWrite(redPin, 100);
 analogWrite(greenPin, 255);
 digitalWrite(bluePin, LOW);
 delay(1000);

 analogWrite(redPin, 50);
 analogWrite(greenPin, 50);
 digitalWrite(bluePin, HIGH);
 delay(1000);
\
```

Code Explanation

- In an Arduino program, first we define the pins or variables. This means we give a name to a pin number so it is easy to use in the code. For example, if we write int ledPin = 5;, it means we are using pin number 5 and calling it ledPin.
- Next comes the setup() function. This part runs only one time when the Arduino is turned on. It is used to prepare the pins and start things like the Serial Monitor. Inside setup, we use pinMode() to tell the Arduino whether a pin will work as an input or an output. For example, setting a pin as OUTPUT allows it to control devices like an LED.
- After that comes the loop() function. This part runs again and again forever, so the Arduino keeps checking or controlling the circuit continuously.
- Inside the loop, we can use digitalWrite() to turn an output ON or OFF. For example, digitalWrite(ledPin, HIGH); will turn the LED ON, and digitalWrite(ledPin, LOW); will turn it OFF. We can also use analogWrite() to control the brightness of an LED or the speed of a motor using PWM. The value ranges from 0 (off) to 255 (full brightness), with values in between giving different levels.

So overall, the program first prepares everything in setup(), then keeps running the loop continuously, controlling devices using functions like pinMode(), digitalWrite(), and analogWrite().

Try This Yourself Code

This Try This Yourself code keeps the same topic of Mood Lamp but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
const int redPin = 5;
const int greenPin = 6;
const int bluePin = 7;

void setup() \
\

void loop() \
 analogWrite(redPin, 255);
 analogWrite(greenPin, 80);
 digitalWrite(bluePin, LOW);
 delay(800);
 analogWrite(redPin, 40);
 analogWrite(greenPin, 255);
 digitalWrite(bluePin, LOW);
 delay(800);
 analogWrite(redPin, 0);
 analogWrite(greenPin, 120);
 digitalWrite(bluePin, HIGH);
 delay(800);
\
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learn

the basic idea of Mood Lamp how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Advanced: Advanced RGB Mixing Logic

Class Explanation

In this class, students learn the basic idea of Advanced RGB Mixing Logic. The code below is kept simple so beginners can focus on understanding the main concept without extra complexity.

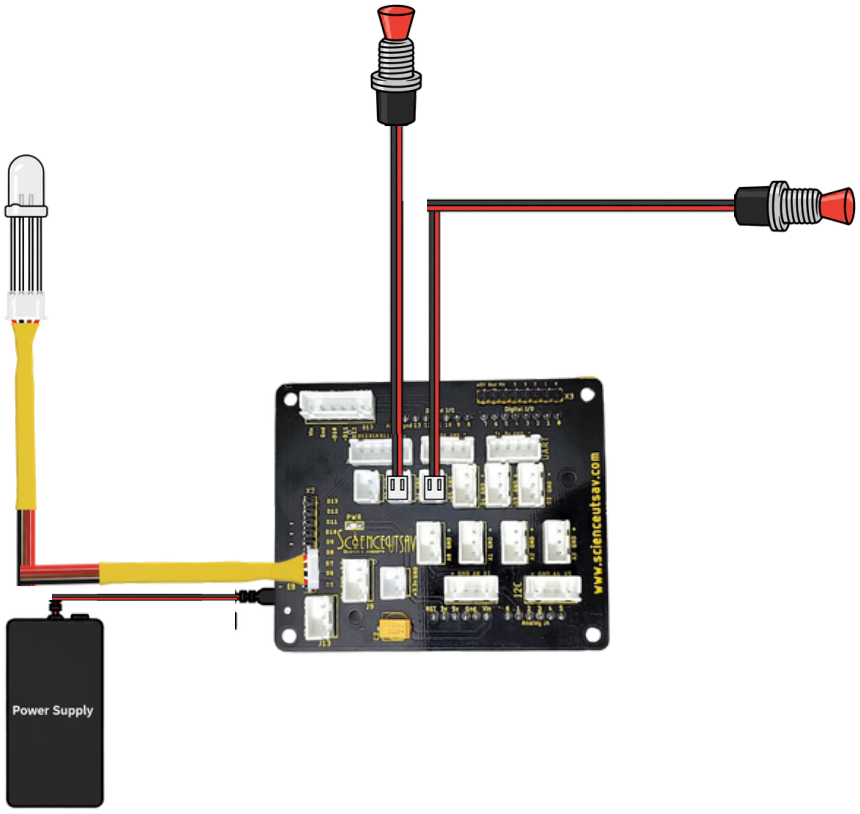
Materials Required

- Arduino with attached shield
- 1 × 4-Leg RGB LED on breakout board
- 2 × Push Button
- RMC wires
- USB cable

ow to Connect

- 1 × Push Button → Arduino pin D2
- 1 × Push Button → Arduino pin D3
- Attach the 4-PIN RMC to Break out pins D5,D6,D7
- Connect using RMC wires with LED :
  - Red leg → Arduino pin D5
  - Green leg → Arduino pin D6
  - Blue leg → Arduino pin D7
- Plug the Arduino into your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
const int button1 = 2;
const int button2 = 3;
const int redPin = 5;
const int greenPin = 6;
const int bluePin = 7;

void setup() \
 pinMode(button1, INPUT_PULLUP);
 pinMode(button2, INPUT_PULLUP);
 pinMode(redPin, OUTPUT);
 pinMode(greenPin, OUTPUT);
 pinMode(bluePin, OUTPUT);
\

void loop() \
 if (digitalRead(button1) == LOW) \
   digitalWrite(redPin, HIGH);
   digitalWrite(greenPin, LOW);
   digitalWrite(bluePin, LOW);
 \ else if (digitalRead(button2) == LOW) \
   digitalWrite(redPin, LOW);
   digitalWrite(greenPin, HIGH);
   digitalWrite(bluePin, LOW);
 \ else \
   digitalWrite(redPin, LOW);
   digitalWrite(greenPin, LOW);
   digitalWrite(bluePin, HIGH);
 \
\
```

Code Explanation

- In an Arduino program, first we define the pins or variables. This means we give a name to a pin number so it is easy to use in the code. For example, if we write int ledPin = 5; and int buttonPin = 2;, it means we are using those pins and giving them simple names.
- Next comes the setup() function. This part runs only one time when the Arduino is turned on. It is used to prepare the pins and start things like the Serial Monitor. Inside setup, we use pinMode() to tell the Arduino whether a pin will work as an input or an output. For example, the LED pin is set as OUTPUT and the button pin is set as INPUT.
- After that comes the loop() function. This part runs again and again forever, so the Arduino keeps checking or controlling the circuit continuously.
- Inside the loop, we use digitalRead() to check the state of a button, switch, or sensor. It tells us whether the input is HIGH (pressed/on) or LOW (not pressed/off). We can then use digitalWrite() to turn an output like an LED ON or OFF.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Advanced RGB Mixing Logic but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
const int button1 = 2;
const int button2 = 3;
const int redPin = 5;
const int greenPin = 6;
const int bluePin = 7;

void setup() \
 pinMode(button1, INPUT_PULLUP);
 pinMode(button2, INPUT_PULLUP);
 pinMode(redPin, OUTPUT);
 pinMode(greenPin, OUTPUT);
 pinMode(bluePin, OUTPUT);
\

void loop() \
 bool firstPressed = digitalRead(button1) == LOW;
 bool secondPressed = digitalRead(button2) == LOW;

 if (firstPressed && secondPressed) \
   digitalWrite(redPin, HIGH);
   digitalWrite(greenPin, HIGH);
   digitalWrite(bluePin, HIGH);
 \ else if (firstPressed) \
   digitalWrite(redPin, HIGH);
   digitalWrite(greenPin, LOW);
   digitalWrite(bluePin, LOW);
 \ else if (secondPressed) \
   digitalWrite(redPin, LOW);
   digitalWrite(greenPin, HIGH);
   digitalWrite(bluePin, LOW);
 \ else \
   digitalWrite(redPin, LOW);
   digitalWrite(greenPin, LOW);
   digitalWrite(bluePin, HIGH);
 \
\
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learn

the basic idea of Mood Lamp how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Expert: PWM Control for Smooth Transitions

Class Explanation

In this class, students learn the basic idea of PWM Control for Smooth Transitions. The code below is kept simple so beginners can focus on understanding the main concept without extra complexity.

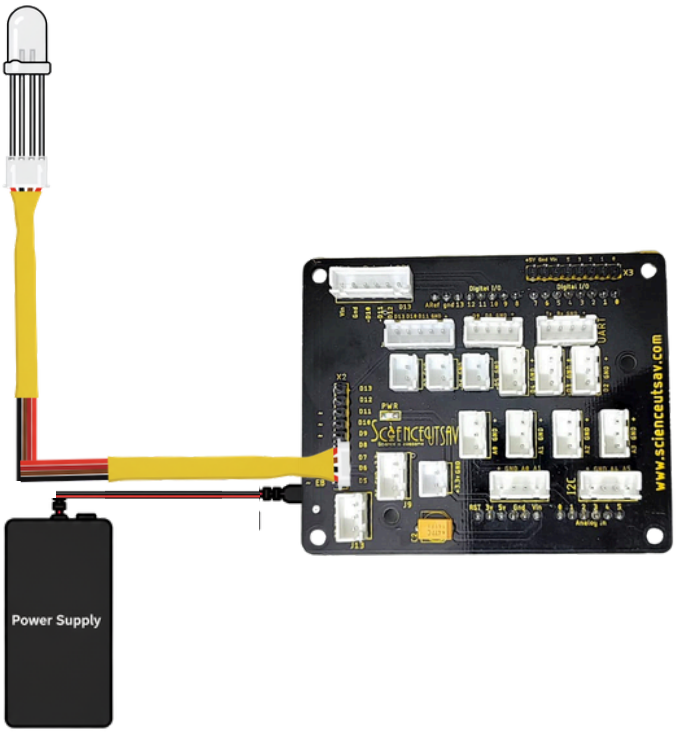
Materials Required

- Arduino with attached shield
- 1 × 4-Leg RGB LED on breakout board
- RMC wires
- USB cable

ow to Connect

- Attach the 4-PIN RMC to Break out pins D5,D6,D7
- Connect using RMC wires with LED :
  - Red leg → Arduino pin D5
  - Green leg → Arduino pin D6
- Plug the Arduino into your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
const int redPin = 5;
const int greenPin = 6;

void setup() \
 pinMode(redPin, OUTPUT);
 pinMode(greenPin, OUTPUT);
\

void loop() \
 for (int value = 0; value <= 255; value += 5) \
   analogWrite(redPin, value);
   analogWrite(greenPin, 255 - value);
   delay(30);
 \
\
```

Code Explanation

- In an Arduino program, first we define the pins or variables. This means we give a name to a pin number so it is easy to use in the code. For example, if we write int ledPin = 9;, it means we are using pin number 9 and calling it ledPin.

- Next comes the setup() function. This part runs only one time when the Arduino is turned on. It is used to prepare the pins and start things like the Serial Monitor. Inside setup, we use pinMode() to tell the Arduino whether a pin will work as an input or an output. For example, setting a pin as OUTPUT allows it to control devices like an LED.

- After that comes the loop() function. This part runs again and again forever, so the Arduino keeps checking or controlling the circuit continuously.

- Inside the loop, we can use analogWrite() to control the brightness of an LED or the speed of a motor using PWM. The values range from 0 (off) to 255 (full brightness), with values in between giving different levels. We can also use a for loop to repeat the same action multiple times without writing the same code again and again, making the program shorter and easier to understand.

- So overall, the program first prepares everything in setup(), then keeps running the loop continuously, controlling devices and repeating actions using functions like pinMode(), analogWrite(), and loops like for.

Try This Yourself Code

This Try This Yourself code keeps the same topic of PWM Control for Smooth Transitions but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
const int redPin = 5;
const int greenPin = 6;

void setup() \
\

void loop() \
 for (int value = 0; value <= 255; value += 10) \
   analogWrite(redPin, value);
   analogWrite(greenPin, 255 - value);
   delay(40);
 \

 for (int value = 255; value >= 0; value -= 10) \
   analogWrite(redPin, value);
   analogWrite(greenPin, 255 - value);
   delay(40);
 \
\
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learn

the basic idea of Mood Lamp how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Learning Outcomes

- students practice the same topic with a small variation instead of starting from zero
- After completing this session, students should be able to:
- the basic idea of RGB LED Basics how the code connects with the circuit behavior how to upload and test a simple Arduino sketch
- the basic idea of Mood Lamp how the code connects with the circuit behavior how to upload and test a simple Arduino sketch
- the basic idea of Advanced RGB Mixing Logic how the code connects with the circuit behavior how to upload and test a simple Arduino sketch
- the basic idea of PWM Control for Smooth Transitions how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Try Out Other Things

- use the T2 Try This Yourself code after each main class code before asking students to build a completely new variation
- ask students to point out the exact lines that changed between the first code and the T2 code
- let students first run the T2 code as it is and then change one value, one condition, or one output